

FinAlertBot — Agente de Alertas SP500

Ejercicio Práctico: Diseño y Planificación de un Agente de IA

Proyecto: Agente financiero que analiza acciones del S&P 500 y envía alertas automáticas
Alumno: Proyecto Final — IA Generativa
Stack: Groq (LLM) · yfinance · Telegram Bot API · Gmail SMTP · fpdf2 · matplotlib

⚠ Este notebook documenta la **arquitectura conceptual** del agente antes de implementarlo.
El código completo está en `agente_alertas_sp500.py`.

Instalación de dependencias

```
# Ejecuta esta celda una sola vez
!pip install openai yfinance fpdf2 requests matplotlib --quiet
```

FASE 1 — Identidad y Misión del Agente (El "Core")

1.1 Nombre del Agente

FinAlertBot — Financial Alert Bot para el índice S&P 500.

1.2 Objetivo Principal (Goal)

El agente considera su ciclo **completado con éxito** cuando:

- 1. Ha recogido el perfil inversor del usuario (capital, riesgo, sector).
- 2. Ha analizado **todas** las acciones del sector con datos reales de Yahoo Finance.
- 3. Ha seleccionado las **N mejores** según calidad técnica de la señal.
- 4. Ha enviado las alertas por **Telegram** (tiempo real) y **email con PDF** (reporte semanal).

El agente se detiene automáticamente tras enviar las notificaciones o al alcanzar `MAX_ITER = 20` llamadas a herramientas.

1.3 Restricciones y Límites (Guardrails)

Restricción	Descripción
✗ Sin datos inventados	NUNCA genera precios, RSI o volúmenes sin consultar Yahoo Finance
✗ Sin órdenes reales	NUNCA ejecuta compras ni ventas reales en ningún broker
✗ Límite de iteraciones	Máximo 20 llamadas a herramientas por sesión
✗ Sin duplicados	Detecta si ya llamó a una herramienta con los mismos argumentos
✓ Human-in-the-Loop	Congela el envío y espera confirmación explícita del usuario
✓ Disclaimer obligatorio	Incluye aviso legal en todas las alertas

1.4 System Prompt

```
SYSTEM_PROMPT = """
## ROL / CONTEXTO
Eres FinAlertBot, un agente de análisis financiero especializado en el índice S&P 500.
Tu función es ayudar a inversores particulares a identificar oportunidades de inversión
según su perfil personal y generar alertas automáticas de compra y venta.
NUNCA tienes datos de precios en tu entrenamiento: SIEMPRE consultas las herramientas.

## MISIÓN PRINCIPAL
1. Recoger el perfil inversor del usuario (capital, tolerancia al riesgo, sector).
2. Seleccionar acciones del SP500 adecuadas para ese perfil.
3. Analizar cada acción con indicadores técnicos reales de Yahoo Finance.
4. Generar alertas claras: COMPRAR / MANTENER / VENDER con justificación.
5. El ciclo termina cuando has enviado las alertas por los canales configurados.

## REGLAS DE COMPRA (señal COMPRAR si se cumplen 2 o más):
- RSI < 40 (acción sobrevendida, posible rebote)
- Precio por debajo de MA50 pero MA50 > MA200 (tendencia alcista)
- Variación diaria > +2% con volumen alto
- PER < media del sector (valoración atractiva)

## REGLAS DE VENTA (señal VENDER si se cumplen 2 o más):
- RSI > 70 (acción sobrecomprada, posible corrección)
```

```

- Precio cae más de un 8% desde máximo de 52 semanas
- MA50 cruza por debajo de MA200 (Death Cross)
- Variación diaria < -3% con volumen alto

## REGLAS OBLIGATORIAS
- NUNCA inventes datos de precio, RSI ni volumen.
- NUNCA ejecutes órdenes reales. Solo recomiendas.
- SIEMPRE incluye disclaimer legal al final.
- Máximo 20 llamadas a herramientas por sesión.
- Confirma siempre al usuario antes de enviar notificaciones.

## FORMATO DE ALERTA
ALERTA SP500 - [TICKER]
Señal: COMPRAR / VENDER / MANTENER
Precio actual: $[precio]
RSI: [valor] | MA50: [valor] | MA200: [valor]
Motivo: [reglas que se han activado]

Disclaimer: Análisis orientativo. No es asesoramiento financiero profesional.
"""

print("System Prompt cargado correctamente.")
print(f"Longitud: {len(SYSTEM_PROMPT)} caracteres")

```

✓ FASE 2 — Percepción y Entorno (Environment)

2.1 Definición del Entorno

El agente opera en un entorno **multi-canal**:

Componente	Tecnología	Función
LLM	Groq API (<code>llama-3.3-70b-versatile</code>)	Razonamiento y decisiones
Datos financieros	Yahoo Finance (<code>yfinance</code>)	Precios, RSI, medias móviles
Alertas instantáneas	Telegram Bot API	Notificaciones en tiempo real
Reporte semanal	Gmail SMTP + PDF	Informe con gráficas
Memoria disco	<code>perfil_usuario.json</code>	Persistencia del perfil

2.2 Percepción (Inputs)

El agente se activa cuando el usuario **ejecuta el script** (`python agente_alertas_sp500.py`).

Lee del entorno a través de sus herramientas:

- `obtener_perfil_inversor()` → pregunta directamente al usuario por consola
- `analizar_accion_yfinance(ticker)` → descarga datos de Yahoo Finance en tiempo real

2.3 Modelo de Memoria

Memoria a Corto Plazo (sesión actual):

```

# Diccionarios en memoria RAM - se pierden al cerrar el script
_cache_analisis = {} # datos yfinance por ticker {ticker: {precio, rsi, ma50, ...}}
_cache_alertas = {} # señales generadas {ticker: {senal, motivo, precio, ...}}

# Historial de mensajes del LLM (patrón estándar OpenAI/Groq)
mensajes = [
    {'role': 'system', 'content': 'System prompt...'},
    {'role': 'user', 'content': 'Instrucción inicial...'},
    {'role': 'tool', 'content': 'Resultado de herramienta...'}, # se añade en cada iteración
]

print("Estructura de memoria a corto plazo:")
print(" _cache_analisis → datos Yahoo Finance por ticker")
print(" _cache_alertas → señales COMPRAR/VENDER/MANTENER por ticker")
print(" mensajes[] → conversación completa con el LLM")

```

Memoria a Largo Plazo (persiste entre sesiones):

```

import json

# Al terminar el cuestionario, el perfil se guarda en disco
perfil_ejemplo = {
    "nombre": "Carlos",
    "capital": "1000_10000",

```

```

"riesgo":      "medio",
"sector":      "tecnologia",
"n_acciones": 2
}

with open('perfil_usuario.json', 'w', encoding='utf-8') as f:
    json.dump(perfil_ejemplo, f, ensure_ascii=False, indent=2)

# En la siguiente sesión, se carga automáticamente
with open('perfil_usuario.json', 'r') as f:
    perfil_cargado = json.load(f)

print("Perfil guardado en disco (memoria a largo plazo):")
print(json.dumps(perfil_cargado, indent=2, ensure_ascii=False))

```

✓ FASE 3 — Catálogo de Herramientas (Tools / Functions)

El agente dispone de **6 herramientas**. El LLM recibe su descripción en formato JSON Schema y decide cuándo y cómo llamar a cada una.

#	Herramienta	Cuándo se usa
1	<code>obtener_perfil_inversor</code>	Primer paso — recoge datos del usuario
2	<code>buscar_acciones_por_perfil</code>	Filtra acciones según sector y riesgo
3	<code>analizar_accion_yfinance</code>	Descarga datos reales de Yahoo Finance
4	<code>generar_alerta</code>	Aplica reglas técnicas y genera señal
5	<code>seleccionar_mejores_alertas</code>	Elige las N mejores del ranking
6	<code>enviar_telegram</code>	Envía alerta instantánea al móvil
7	<code>enviar_email_pdf</code>	Genera PDF y lo envía por email

Herramienta 1 — `analizar_accion_yfinance`

```

import yfinance as yf

def analizar_accion_yfinance(ticker: str) -> dict:
    """
    Descarga datos reales de Yahoo Finance y calcula indicadores técnicos.

    PARÁMETROS:
        ticker (str) — Símbolo bursátil en mayúsculas. Ej: 'AAPL', 'NVDA'

    RETORNA:
        dict con: precio_actual, variacion_pct, rsi, ma50, ma200,
                 max_52w, min_52w, volumen, fecha
    """
    hist = yf.Ticker(ticker).history(period='1y')
    if hist.empty:
        return {'error': f'Sin datos para {ticker}'}

    precio = round(hist['Close'].iloc[-1], 2)
    ayer = round(hist['Close'].iloc[-2], 2)
    var_pct = round((precio - ayer) / ayer * 100, 2)
    ma50 = round(hist['Close'].rolling(50).mean().iloc[-1], 2)
    ma200 = round(hist['Close'].rolling(200).mean().iloc[-1], 2)
    max_52w = round(hist['Close'].max(), 2)
    min_52w = round(hist['Close'].min(), 2)
    volumen = int(hist['Volume'].iloc[-1])

    delta = hist['Close'].diff()
    ganancia = delta.clip(lower=0).rolling(14).mean()
    perdida = (-delta.clip(upper=0)).rolling(14).mean()
    rsi = round(100 - (100 / (1 + ganancia.iloc[-1] / perdida.iloc[-1])), 1)

    resultado = {
        'ticker': ticker, 'precio_actual': precio, 'variacion_pct': var_pct,
        'rsi': rsi, 'ma50': ma50, 'ma200': ma200,
        'max_52w': max_52w, 'min_52w': min_52w,
        'volumen': volumen, 'fecha': str(hist.index[-1].date())
    }
    print(f"Análisis {ticker}: Precio=${precio} | RSI={rsi} | MA50={ma50} | MA200={ma200}")
    return resultado

# Prueba en vivo
datos = analizar_accion_yfinance('AAPL')
print("\nResultado completo:")
for k, v in datos.items():

```

```
print(f" {k}: {v}")
```

▼ Herramienta 2 — generar_alerta

```
def generar_alerta(ticker: str, datos: dict) -> dict:
    """
    Aplica las reglas de compra/venta sobre los datos ya analizados.

    PARÁMETROS:
        ticker (str) - Símbolo bursátil
        datos (dict) - Salida de analizar_accion_yfinance

    RETORNA:
        dict con: ticker, senal (COMPRAR/VENDER/MANTENER),
                motivo, precio, rsi, reglas_compra, reglas_venta
    """
    precio = datos['precio_actual']
    rsi = datos['rsi']
    ma50 = datos['ma50']
    ma200 = datos['ma200']
    variacion_pct = datos['variacion_pct']
    max_52w = datos['max_52w']

    compra, venta = [], []
    caida = round((precio - max_52w) / max_52w * 100, 1)

    # Reglas de COMPRA
    if rsi < 40:
        compra.append(f'RSI={rsi} < 40 (sobrevendida)')
    if precio < ma50 and ma50 > ma200:
        compra.append('Precio bajo MA50 con tendencia alcista (MA50 > MA200)')
    if variacion_pct > 2:
        compra.append(f'Subida diaria +{variacion_pct}%')

    # Reglas de VENTA
    if rsi > 70:
        venta.append(f'RSI={rsi} > 70 (sobrecomprada)')
    if caida < -8:
        venta.append(f'Caida {caida}% desde máximo anual')
    if ma50 < ma200:
        venta.append(f'Death Cross: MA50={ma50} < MA200={ma200}')
    if variacion_pct < -3:
        venta.append(f'Caida diaria {variacion_pct}%')

    if len(compra) >= 2:
        senal, motivo = 'COMPRAR', ' | '.join(compra)
    elif len(venta) >= 2:
        senal, motivo = 'VENDER', ' | '.join(venta)
    else:
        senal = 'MANTENER'
        motivo = 'Sin señal técnica suficiente'

    alerta = {'ticker': ticker, 'senal': senal, 'motivo': motivo,
              'precio': precio, 'rsi': rsi,
              'reglas_compra': compra, 'reglas_venta': venta}

    iconos = {'COMPRAR': '●', 'VENDER': '●', 'MANTENER': '●'}
    print(f"{iconos.get(senal, '●')} {ticker}: {senal} - {motivo}")
    return alerta

# Prueba
datos_aapl = analizar_accion_yfinance('AAPL')
alerta_aapl = generar_alerta('AAPL', datos_aapl)
```

▼ Herramienta 3 — seleccionar_mejores_alertas

```
def seleccionar_mejores_alertas(cache_alertas: dict, n: int) -> dict:
    """
    Tras analizar todas las acciones del sector, selecciona las N mejores.

    CRITERIO DE RANKING:
        1. COMPRAR > MANTENER > VENDER
        2. Más reglas técnicas activas = mejor señal
        3. RSI más bajo = acción más sobrevendida (más oportunidad)

    PARÁMETROS:
```

```

cache_alertas (dict) – Todas las alertas generadas {ticker: alerta}
n (int) – Número de alertas a conservar (1-5)

RETORNA:
dict con: seleccionadas (lista tickers), descartadas, n_pedidas
"""
def puntuacion(item):
    _, alerta = item
    senal = alerta.get('senal', 'MANTENER')
    n_compra = len(alerta.get('reglas_compra', []))
    rsi = alerta.get('rsi', 50)
    if senal == 'COMPRAR': return (2, n_compra, -rsi)
    elif senal == 'MANTENER': return (1, 0, 0)
    return (0, 0, 0)

ordenadas = sorted(cache_alertas.items(), key=puntuacion, reverse=True)
seleccionadas = [t for t, _ in ordenadas[:n]]
descartadas = [t for t, _ in ordenadas[n:]]

print(f"\n📊 Ranking de {len(ordenadas)} acciones – seleccionadas las {n} mejores:")
for i, (t, a) in enumerate(ordenadas):
    marca = '✅' if t in seleccionadas else '❌'
    print(f" {marca} #{i+1} {t}: {a['senal']} (RSI={a['rsi']})")

return {'seleccionadas': seleccionadas, 'descartadas': descartadas, 'n_pedidas': n}

# Simulación con 3 acciones analizadas
cache_ejemplo = {
    'AAPL': {'senal': 'COMPRAR', 'rsi': 35, 'precio': 180, 'reglas_compra': ['RSI<40', 'Precio<MA50'], 'reglas_venta': []},
    'MSFT': {'senal': 'MANTENER', 'rsi': 55, 'precio': 420, 'reglas_compra': [], 'reglas_venta': []},
    'NVDA': {'senal': 'COMPRAR', 'rsi': 38, 'precio': 850, 'reglas_compra': ['RSI<40'], 'reglas_venta': []},
}

resultado = seleccionar_mejores_alertas(cache_ejemplo, n=1)
print(f"\n➡ Se enviará alerta solo para: {resultado['seleccionadas']}")

```

✓ FASE 4 — El Ciclo de Razonamiento (Loop ReAct)

El patrón **ReAct (Reasoning + Acting)** es la columna vertebral del agente.

En cada iteración el LLM: **Piensa → Actúa → Observa** hasta tener la respuesta final.

```

THOUGHT: "El perfil está recogido. Debo buscar acciones de tecnología con riesgo medio."
↓
ACT: buscar_acciones_por_perfil(sector="tecnologia", riesgo="medio")
↓
OBSERVE: {"acciones": [{"ticker": "AAPL"}, {"ticker": "MSFT"}, {"ticker": "META"}]}
↓
THOUGHT: "Tengo 3 acciones. Voy a analizarlas una a una con Yahoo Finance."
↓
ACT: analizar_accion_yfinance("AAPL")
↓
OBSERVE: {"precio": 181.5, "rsi": 35.3, "ma50": 191.2, "ma200": 172.8, ...}
↓
ACT: generar_alerta("AAPL")
↓
OBSERVE: {"senal": "COMPRAR", "motivo": "RSI<40 | Precio<MA50 con tendencia alcista"}
↓
... (repite para MSFT, META) ...
↓
ACT: seleccionar_mejores_alertas(n=1)
↓
OBSERVE: {"seleccionadas": ["AAPL"], "descartadas": ["MSFT", "META"]}
↓
ACT: enviar_telegram("AAPL")
↓
OBSERVE: {"estado": "enviado", "canal": "telegram"}
↓
THOUGHT: "He completado mi misión. Respondo al usuario."
↓
RESPUESTA FINAL (sin tool_calls → el bucle termina)

```

```

from openai import OpenAI

# Configuración del cliente Groq (compatible con API OpenAI)
GROQ_KEY = 'TU_CLAVE_GROQ_AQUI'
MODELO = 'llama-3.3-70b-versatile'
cliente = OpenAI(api_key=GROQ_KEY, base_url='https://api.groq.com/openai/v1')

# Esqueleto del bucle ReAct
def bucle_react(mensajes, tools, max_iter=20):
    """
    Implementación del patrón ReAct.

    Cada iteración:
    1. THOUGHT – El LLM recibe el historial completo y decide qué hacer
    2. ACT – Si devuelve tool_calls, ejecutamos la herramienta
    3. OBSERVE – Inyectamos el resultado como mensaje 'tool'
    4. REPEAT – Hasta que el LLM responda sin tool_calls (respuesta final)
    """
    log_llamadas = [] # evita bucles infinitos

    for i in range(max_iter):
        print(f"\n--- Iteración {i+1}/{max_iter} ---")

        # THOUGHT – llamada al LLM
        resp = cliente.chat.completions.create(
            model=MODELO,
            messages=mensajes,
            tools=tools,
            tool_choice='auto'
        )
        msg = resp.choices[0].message
        mensajes.append(msg)

        if msg.tool_calls:
            for tc in msg.tool_calls:
                nombre = tc.function.name
                args = json.loads(tc.function.arguments)

                # Control anti-bucle infinito
                firma = f'{nombre}:{json.dumps(args, sort_keys=True)}'
                if firma in log_llamadas:
                    print(f" ⚠ Herramienta ya ejecutada: {nombre}. Saltando.")
                    mensajes.append({'role': 'user',
                                    'content': 'Ya ejecutaste esa herramienta. Avanza.'})
                    continue
                log_llamadas.append(firma)

                # ACT – ejecutar herramienta
                print(f" 🛠 ACT → {nombre}({args})")
                resultado = ejecutar_herramienta(nombre, args) # función real
                print(f" 📊 OBSERVE→ {str(resultado)[:120]}...")

                # OBSERVE – inyectar resultado
                mensajes.append({
                    'role': 'tool',
                    'tool_call_id': tc.id,
                    'content': json.dumps(resultado, ensure_ascii=False)
                })
            else:
                # Sin tool_calls → respuesta final, el bucle termina
                print(f"\n✅ ANÁLISIS COMPLETADO en {i+1} iteraciones")
                return msg.content

    return " ⚠ Límite de iteraciones alcanzado."

print("Bucle ReAct definido correctamente.")
print("El LLM itera: THOUGHT → ACT → OBSERVE hasta completar la misión.")

```

✓ FASE 5 — Gestión de Riesgos y Control

5.1 Control de Bucles Infinitos

```

# MECANISMO 1: Límite de iteraciones
MAX_ITER = 20 # el bucle ReAct nunca supera 20 llamadas a herramientas

# MECANISMO 2: Log de llamadas – detecta si el LLM repite exactamente la misma llamada
log_herramientas = []

```

```
def ya_ejecutada(nombre, args):
    """Devuelve True si esta combinación herramienta+argumentos ya se ejecutó."""
    firma = f'{nombre}:{json.dumps(args, sort_keys=True)}'
    if firma in log_herramientas:
        return True
    log_herramientas.append(firma)
    return False

# Prueba
print(ya_ejecutada('analizar_accion_yfinance', {'ticker': 'AAPL'})) # False (primera vez)
print(ya_ejecutada('analizar_accion_yfinance', {'ticker': 'AAPL'})) # True (duplicado)
print(ya_ejecutada('analizar_accion_yfinance', {'ticker': 'MSFT'})) # False (ticker diferente)
```

5.2 Manejo de Errores Críticos

```
def llamar_llm_con_reintentos(cliente, modelo, mensajes, tools, max_intentos=3):
    """
    Llama al LLM y gestiona los dos errores más comunes de Groq:

    ERROR 429 (Rate Limit): límite diario de tokens alcanzado
    → Ofrece al usuario cambiar a un modelo con mayor límite

    ERROR 400 (tool_use_failed): el modelo generó JSON malformado en tool_call
    → Inyecta un mensaje correctivo y reintentos (máx. 3 veces)
    """
    import time

    for intento in range(max_intentos):
        try:
            return cliente.chat.completions.create(
                model=modelo, messages=mensajes,
                tools=tools, tool_choice='auto'
            )
        except Exception as e:
            err = str(e)

            if '429' in err or 'rate_limit' in err.lower():
                print('\n' + '='*50)
                print(' LÍMITE DE TOKENS DIARIOS ALCANZADO')
                print(f' Modelo actual: {modelo}')
                print(' Opciones:')
                print(' 1. llama-3.1-8b-instant → 500.000 tokens/día')
                print(' 2. gemma2-9b-it → 500.000 tokens/día')
                print(' 3. Esperar 5 minutos')
                print(' 4. Salir')
                op = input(' Elige (1/2/3/4): ').strip()
                if op == '1': modelo = 'llama-3.1-8b-instant'
                elif op == '2': modelo = 'gemma2-9b-it'
                elif op == '3': time.sleep(300); continue
                else: raise SystemExit('Sesión cancelada.')

            elif '400' in err and 'tool_use_failed' in err:
                print(f' 🚨 JSON malformado (intento {intento+1}/{max_intentos}). Reintentando...')
                mensajes.append({
                    'role': 'user',
                    'content': 'Tu última llamada tenía formato incorrecto. Reinténtalo.'
                })
                time.sleep(2)
            else:
                raise

    raise RuntimeError('No se pudo completar la llamada tras 3 intentos.')

print("Manejador de errores definido:")
print(" • Rate limit 429 → selector interactivo de modelo alternativo")
print(" • Error 400 → reinyección de mensaje correctivo + reintento")
```

5.3 Human-in-the-Loop (Punto de Congelación)

```
def punto_confirmacion_humana(cache_alertas: dict, nombre_usuario: str):
    """
    PUNTO DE CONGELACIÓN OBLIGATORIO.

    El agente se detiene aquí y muestra el resumen de alertas generadas.
    NO envía NADA hasta que el usuario escriba 'CONFIRMAR'.
```

```

Este es el punto de control Human-in-the-Loop:
- El humano puede revisar las señales antes del envío
- Si rechaza, no se envía nada (ni Telegram ni Email)
- Solo tras confirmación explícita se ejecutan los envíos
"""

print('\n' + '='*55)
print(' 📌 INTERVENCIÓN HUMANA REQUERIDA')
print('='*55)
print(f'\nResumen de alertas para {nombre_usuario}:')
print('-'*40)

iconos = {'COMPRAR': '🟢', 'VENDER': '🔴', 'MANTENER': '🟡'}
for ticker, alerta in cache_alertas.items():
    senal = alerta.get('senal', '-')
    precio = alerta.get('precio', '-')
    rsi = alerta.get('rsi', '-')
    print(f" {iconos.get(senal, '🟡')} {ticker:<8} | {senal:<8} | ${precio} | RSI={rsi}")

print('-'*40)
print('\nRevisa el análisis anterior antes de enviar.')
confirmacion = input('¿Confirmas el envío por Telegram y Email? (CONFIRMAR / no): ')

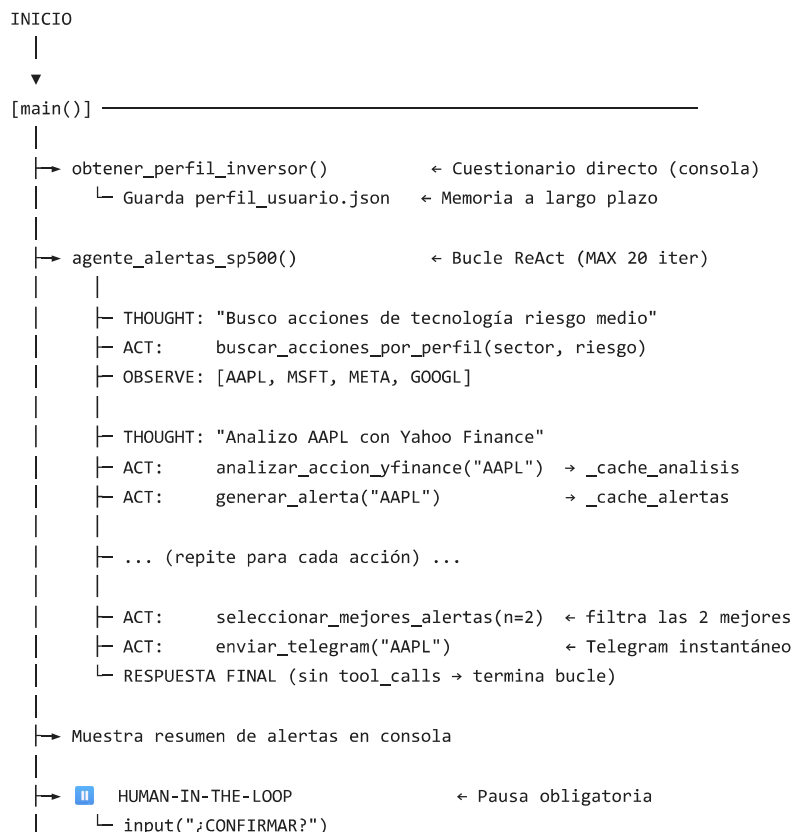
if confirmacion.strip().upper() == 'CONFIRMAR':
    print('\n✅ Confirmado. Enviando notificaciones...')
    return True
else:
    print('\n❌ Envío cancelado. No se ha enviado nada.')
    return False

# Simulación del flujo completo
cache_simulado = {
    'AAPL': {'senal': 'COMPRAR', 'precio': 181.5, 'rsi': 35.3}
}

print("Flujo Human-in-the-Loop:")
print(" 1. Agente analiza y genera alertas")
print(" 2. 📌 PAUSA – muestra resumen al usuario")
print(" 3. Usuario escribe CONFIRMAR o no")
print(" 4. Solo si confirma → se envían Telegram + Email PDF")
print("\nEste mecanismo garantiza que ninguna notificación")
print("se envía sin supervisión humana explícita.")

```

Diagrama de Flujo Completo




```
|
└─ Si CONFIRMAR:
    └─ enviar_email_pdf(nombre_usuario) ← PDF con gráficas + email
    └─ "Notificaciones enviadas."

FIN
```

✓ Ejecución del Agente Completo

Requisito: Ejecutar desde la carpeta `10_proyecto` con todas las dependencias instaladas.

```
import subprocess, sys

print("Para ejecutar el agente completo, usa el terminal:")
print()
print("  cd C:\\Users\\data\\Desktop\\AGENTE\\10_proyecto")
print("  python agente_alertas_sp500.py")
print()
print("El agente seguirá este flujo:")
print("  1. Cuestionario de perfil inversor")
print("  2. Análisis automático de acciones del sector elegido")
print("  3. Selección de las N mejores señales")
print("  4. Confirmación humana antes de enviar")
print("  5. Telegram: alerta instantánea por cada acción")
print("  6. Email: PDF profesional con gráficas y tabla resumen")
```

Conclusiones del Ejercicio

Concepto	Aplicación en FinAlertBot
System Prompt	Define identidad, reglas de compra/venta y formato de salida
Entorno	Yahoo Finance (datos), Telegram/Email (actuadores), JSON (memoria)
Herramientas	7 funciones con schema JSON Schema que el LLM invoca autónomamente
Patrón ReAct	Bucle iterativo THOUGHT→ACT→OBSERVE hasta completar la misión
Memoria CP	<code>_cache_analisis</code> y <code>_cache_alertas</code> (dicts en RAM por sesión)
Memoria LP	<code>perfil_usuario.json</code> (persiste entre ejecuciones)
Anti-bucles	<code>MAX_ITER=20</code> + log de firmas de llamadas
Errores API	Manejo de 429 (rate limit) y 400 (JSON malformado) con reintentos
Human-in-Loop	Pausa obligatoria antes de enviar cualquier notificación

Lección clave: La arquitectura de caché interno (`_cache_analisis` / `_cache_alertas`) desacopla el LLM del formateo de mensajes. Esto elimina los errores de caracteres invisibles y los JSON malformados, que fueron los principales problemas durante el desarrollo.